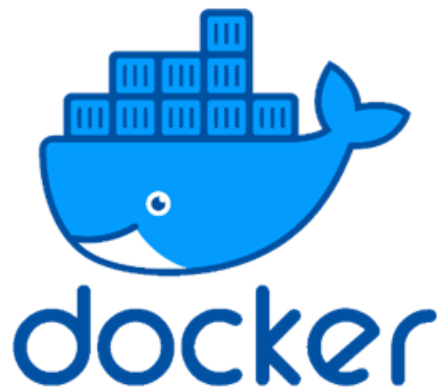


Tarea FFEOE .

Roberto Moreno Moreno | DAW- DAW a distancia | 12/05/2026



Contenido

Ejercicio 1.-	3
Actividad 1.1.-	3
Actividad 1.2.-	4
Actividad 1.3.-	6

Ejercicio 1.-

Este ejercicio relacionado con generación de documentación consistirá en los siguientes apartados :

Actividad 1.1.-

Cita y explica los aspectos más importantes de la herramienta phpdocumentor, así como las etiquetas principales que se usan.

Aspectos más importantes de phpDocumentor: Una buena documentación facilita, en gran medida, el mantenimiento futuro de la aplicación. PhpDocumentor es una de las herramientas más utilizadas para generar documentación de forma automática a partir del código fuente en PHP. Sus características principales son:

- **Formatos de salida:** Permite generar la documentación en varios formatos, destacando HTML (a través de plantillas), PDF y XML (DocBook).
- **Flexibilidad de ejecución:** Se puede ejecutar desde línea de comandos (php CLI), desde una interfaz web, o incluso integrando su funcionalidad dentro de scripts propios al estar desarrollado en PHP.
- **Estructura de la documentación:** En phpDocumentor la documentación se distribuye en bloques llamados "DocBlock". Estos bloques siempre se colocan justo antes del elemento al que documentan (como funciones, clases o variables) y su formato de comentario debe comenzar con / y terminar con */.

Etiquetas principales (Tags): Dentro de cada DocBlock se incluyen marcas o etiquetas que phpDocumentor interpreta con un significado especial. Para nuestro proyecto, destacamos las siguientes:

- **@author:** Se utiliza para indicar el autor del código.
- **@version:** Indica la versión actual del elemento.
- **@param:** Sirve para documentar los parámetros de entrada que recibe una función.
- **@return:** Especifica el valor devuelto por una función.
- **@internal:** Se usa para incluir información que no debería aparecer en la documentación pública, pero sí está disponible como documentación interna exclusiva para desarrolladores.
- **@access:** Permite controlar la visibilidad. Si se marca como 'private', no se genera documentación pública para ese elemento, lo cual es muy útil para ocultar la implementación.

Actividad 1.2.-

Ejecuta la herramienta phpdocumentor en un contenedor Docker. Crea un script PHP con el nombre practica-XXXXXX.php, donde XXXXXX será tu apellido. A continuación escribe dentro de este script bloques de código y DocBlocks para que luego se pueda generar la documentación correspondiente. El script debe contener al menos dos funciones documentadas, indicando mediante las etiquetas vistas en la unidad los siguientes elementos:

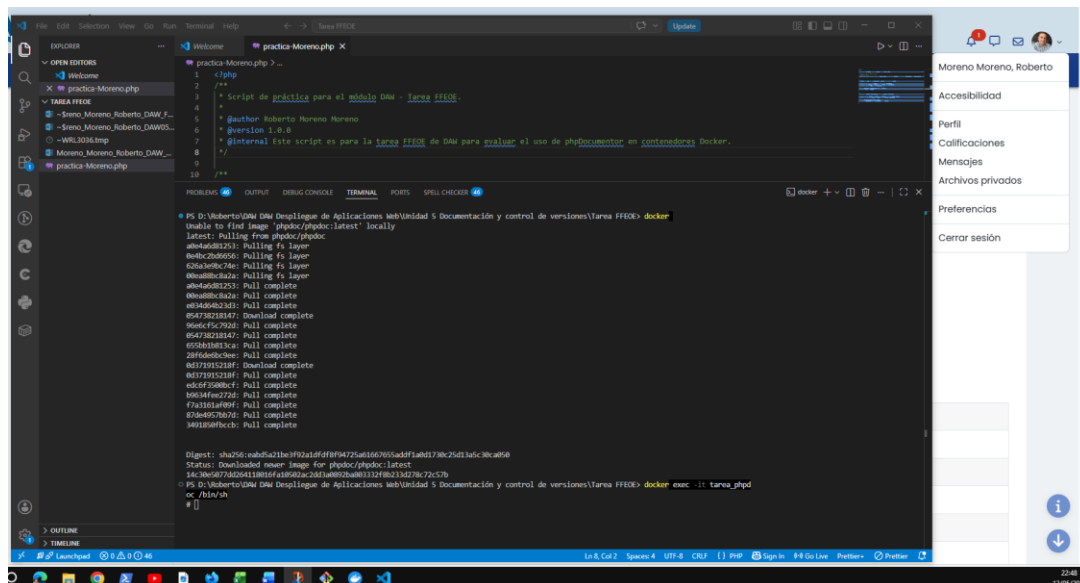
- Parámetros de entrada de la función.
- Parámetros de devuelve la función.
- Autor y versión del script.
- Una anotación que solo sea visible en la documentación para desarrolladores.

Nota: Hay que adjuntar el fichero practica-XXXXXX.php que realices en el directorio .zip.

Seguindo las indicaciones de la tarea, he procedido a ejecutar phpDocumentor utilizando su imagen oficial en Docker (phpdoc/phpdoc). Para ello, he lanzado el contenedor en segundo plano con una consola interactiva mediante el siguiente comando en nuestro terminal:

```
docker run -dit --name tarea_phpdoc --entrypoint /bin/sh phpdoc/phpdoc
```

Una vez creado, he accedido al interior del contenedor mediante: **docker exec -it tarea_phpdoc /bin/sh**



Creación del script PHP documentado: Dentro del entorno de trabajo (Visual Studio Code), he creado el fichero solicitado con el nombre practica-Moreno.php. Según el apartado 2.1 del temario, la documentación se distribuye en bloques DocBlock que comienzan por /.

A continuación, se muestra el código fuente del script cumpliendo estrictamente con todos los requisitos de etiquetas (@author, @version, @internal, @param y @return):

```

<?php
/**
 * Proyecto de práctica para el módulo DAW.
 *
 * @package TareaFFEOE
 * @author Roberto Moreno Moreno
 * @version 1.0.0
 */

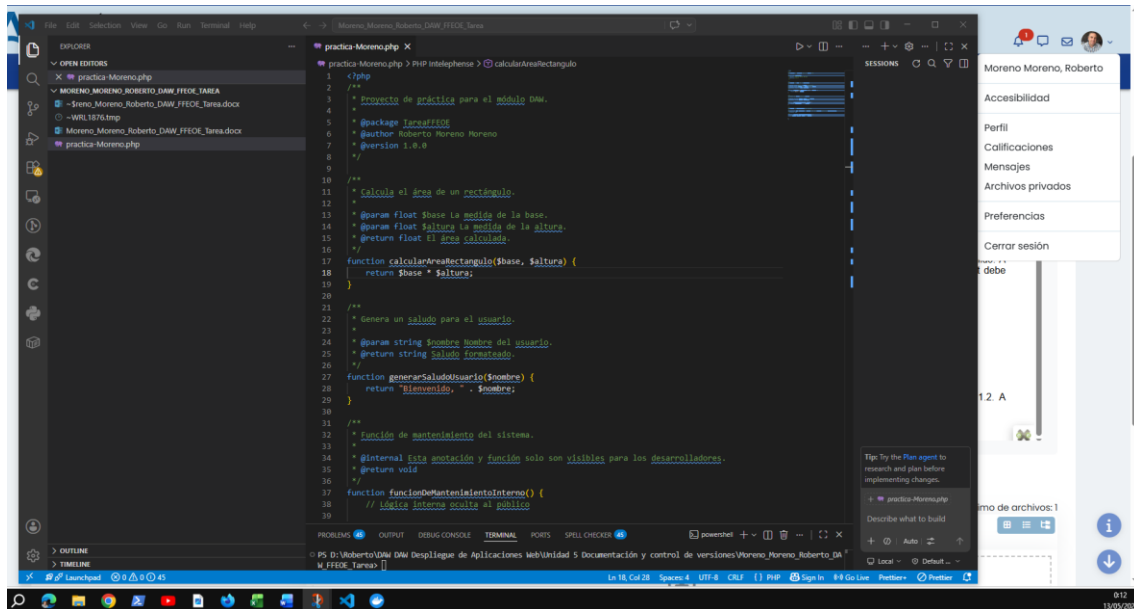
/**
 * Calcula el área de un rectángulo.
 *
 * @param float $base La medida de la base.
 * @param float $altura La medida de la altura.
 * @return float El área calculada.
 */
function calcularAreaRectangulo($base, $altura) {
    return $base * $altura;
}

/**
 * Genera un saludo para el usuario.
 *
 * @param string $nombre Nombre del usuario.
 * @return string Saludo formateado.
 */
function generarSaludoUsuario($nombre) {
    return "Bienvenido, " . $nombre;
}

/**
 * Función de mantenimiento del sistema.
 *
 * @internal Esta anotación y función solo son visibles para los
desarrolladores.
 * @return void
 */
function funcionDeMantenimientoInterno() {
    // Lógica interna oculta al público
}
?>

```

Tuve que cambiar el archivo ya que coloqué el @internal en el global y no me salía la información en la documentación en el html, además tuve que incluir una tercera función para probar @internal y que me aparecieran dos en el html.



Actividad 1.3.-

Después de revisar la documentación, especialmente el apartado 1.2, genera la documentación del script PHP que hemos creado en la actividad 1.2. A continuación muestra el árbol de carpetas y archivos que genera como salida.

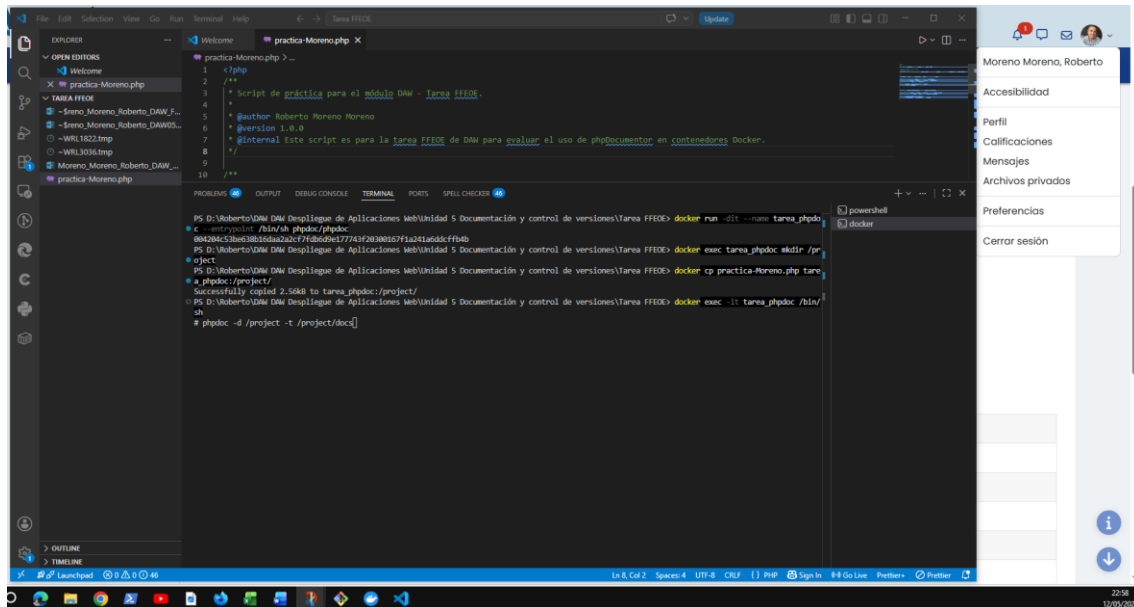
Para generar la documentación, ejecutamos el comando interno de phpDocumentor indicando el directorio de origen (-d) donde está nuestro archivo y el directorio de destino (-t) donde queremos que se guarde la documentación en HTML, tal y como se indica en el apartado 2.3 del tema.

phpdoc -d /project -t /project/docs

Visualización del árbol de directorios: Atendiendo a las recomendaciones del profesor, he instalado la herramienta tree en el sistema para poder visualizar gráficamente la estructura generada.

Ejecuto el comando sobre la carpeta generada: **tree /project/docs**

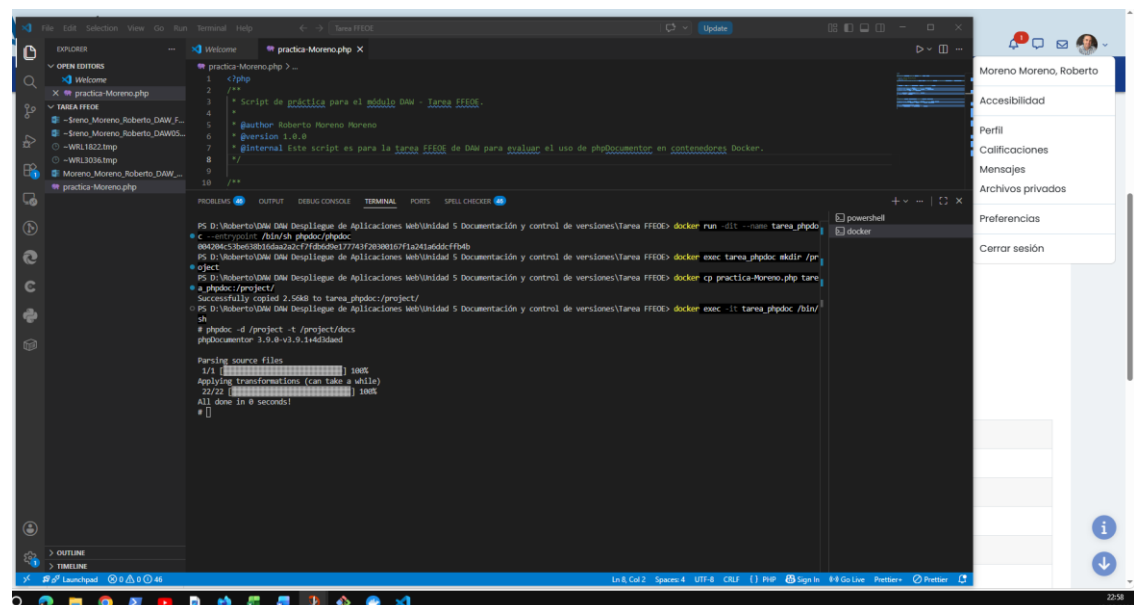
En un primer intento, lancé la secuencia de comandos para levantar el contenedor, crear el directorio de trabajo interno (/project), copiar el script practica-Moreno.php desde la máquina local al interior del contenedor y acceder a la consola interactiva.



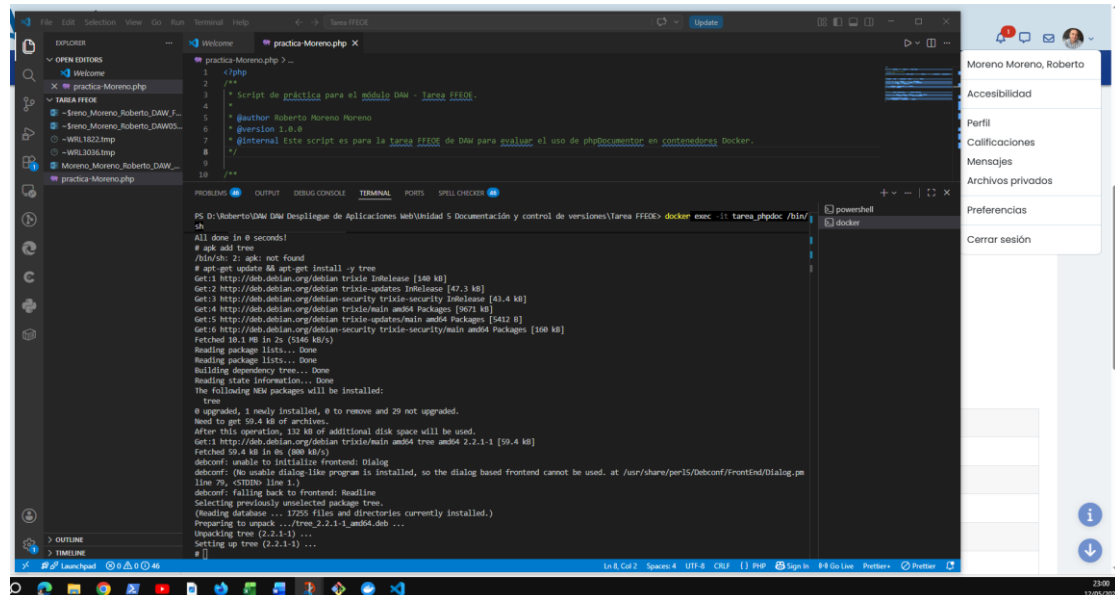
Durante esta primera ejecución, el *daemon* de Docker arrojó un error indicando que el nombre del contenedor `tarea_phpdoc` ya estaba en uso por una ejecución anterior. Para garantizar un entorno de trabajo limpio y evitar conflictos, procedí a eliminar el contenedor residual forzando su borrado mediante el comando: `docker rm -f tarea_phpdoc`

Una vez liberado el nombre, volví a lanzar con éxito la secuencia de comandos para crear el contenedor, preparar el directorio, copiar el script PHP y entrar al terminal.

Estando dentro del contenedor, ejecuté phpDocumentor por línea de comandos. Tal y como se indica en el material didáctico del módulo, la herramienta requiere especificar la ruta de origen de los proyectos con el parámetro `-d` y la carpeta donde se almacenarán los archivos de documentación generada con el parámetro `-t`



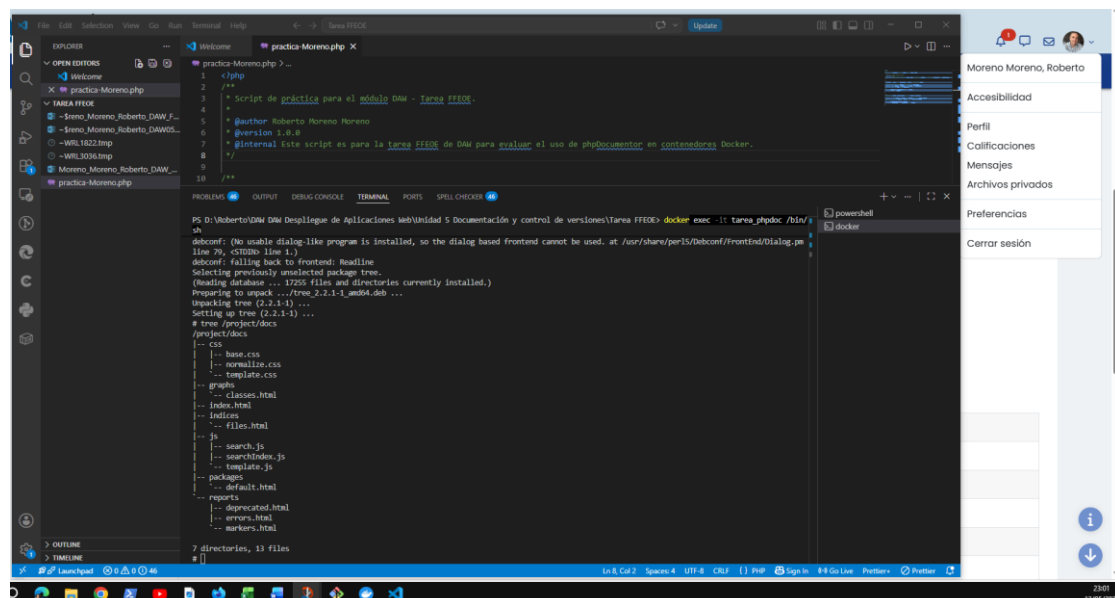
Para cumplir con el requisito de la Actividad 1.3 de mostrar el árbol de carpetas generado, procedí a instalar la utilidad tree dentro del contenedor. Inicialmente intenté usar el gestor de paquetes de Alpine Linux (apk), pero el sistema devolvió un error de comando no encontrado. Al deducir que la imagen Docker de phpDocumentor estaba basada en Debian/Ubuntu (igual que la máquina virtual Squeeze de nuestros apuntes), procedí a actualizar los repositorios e instalar la herramienta usando el gestor de paquetes apt-get.



```

PS D:\Vuberto\DAW Despliegue de Aplicaciones Web\Unidad 5 Documentación y control de versiones\Tarea FFEDE> docker exec -it tarea_phpdoc /bin/sh
All done in 8 seconds!
# apt add tree
/bin/sh: 2: apt: not found
# apt-get update && apt-get install -y tree
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [42.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [41.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5442 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [108 kB]
Fetched 10.1 MB in 7s (5146 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
tree
0 upgraded, 1 newly installed, 0 to remove and 29 not upgraded.
Need to get 59.4 kB of archives.
After this operation, 112 kB of additional disk space will be used.
Get:1 http://deb.debian.org/debian trixie/main amd64 tree amd64 2.2.1-1 [59.4 kB]
Fetched 59.4 kB in 0s (880 kB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm
line 79, <chomo line 1.>)
debconf: falling back to frontend: Readline
Selecting previously unselected package tree.
(Reading database ... 17255 files and directories currently installed.)
Preparing to unpack .../tree_2.2.1-1_amd64.deb ...
Unpacking tree (2.2.1-1) ...
Setting up tree (2.2.1-1) ...
#
  
```

Finalmente, ejecuté el comando tree sobre la carpeta de destino (/project/docs) para verificar la estructura de archivos generada por phpDocumentor. Como se puede observar, se han creado correctamente las subcarpetas de recursos (CSS, JS) y el archivo principal de la documentación (index.html).



```

PS D:\Vuberto\DAW Despliegue de Aplicaciones Web\Unidad 5 Documentación y control de versiones\Tarea FFEDE> docker exec -it tarea_phpdoc /bin/sh
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm
line 79, <chomo line 1.>)
debconf: falling back to frontend: Readline
Selecting previously unselected package tree.
(Reading database ... 17255 files and directories currently installed.)
Preparing to unpack .../tree_2.2.1-1_amd64.deb ...
Unpacking tree (2.2.1-1) ...
Setting up tree (2.2.1-1) ...
# tree /project/docs
/project/docs
|-- css
|   |-- base.css
|   |-- normalize.css
|   |-- template.css
|   |-- ...
|-- graphs
|   |-- classes.html
|   |-- index.html
|   |-- indices
|   |-- files.html
|   |-- ...
|-- js
|   |-- search.js
|   |-- searchIndex.js
|   |-- template.js
|   |-- packages
|   |-- default.html
|   |-- reports
|   |-- deprecated.html
|   |-- errors.html
|   |-- markers.html
|-- ...
7 directories, 13 files
#
  
```


Para comprobar el resultado final, he extraído los archivos del contenedor Docker y visualizado el archivo index.html en un navegador web. En la imagen se puede apreciar cómo phpDocumentor ha interpretado correctamente las etiquetas @author, @version y las descripciones de las funciones, generando una interfaz de usuario navegable y profesional.

